

Slicing Is All You Need

Towards a Universal One-Sided Algorithm for
Distributed Matrix Multiplication

Benjamin Brock

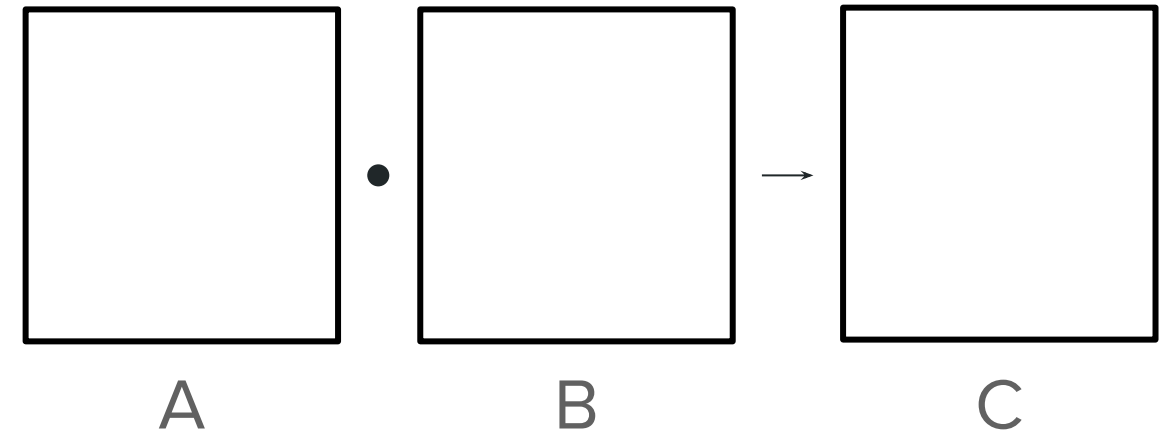
November 16, 2025

Distributed Matrix Multiplication

When a matrix is **too big** to fit on a **single node/GPU**

When computation **takes too long**

⇒ **Partition** matrix across multiple GPUs

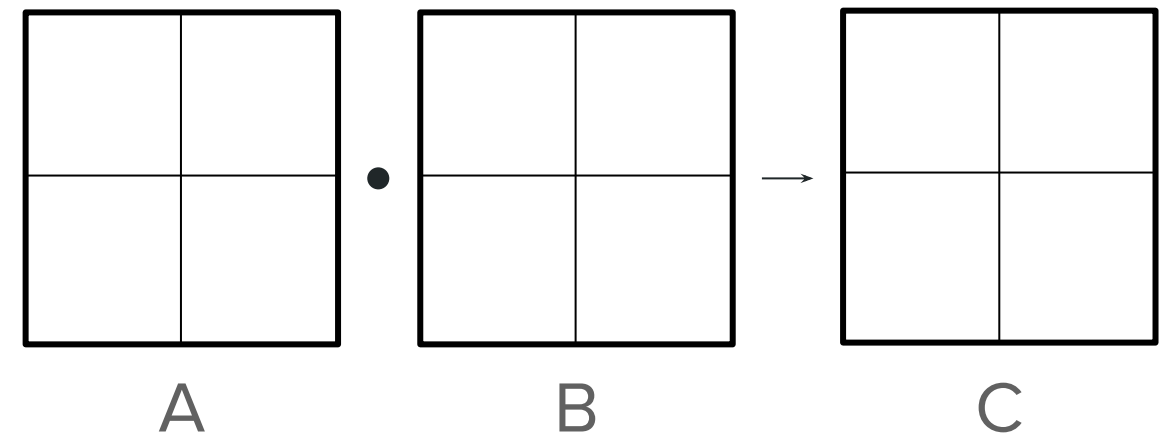


Distributed Matrix Multiplication

When a matrix is **too big** to fit on a **single node/GPU**

When computation **takes too long**

⇒ **Partition** matrix across multiple GPUs

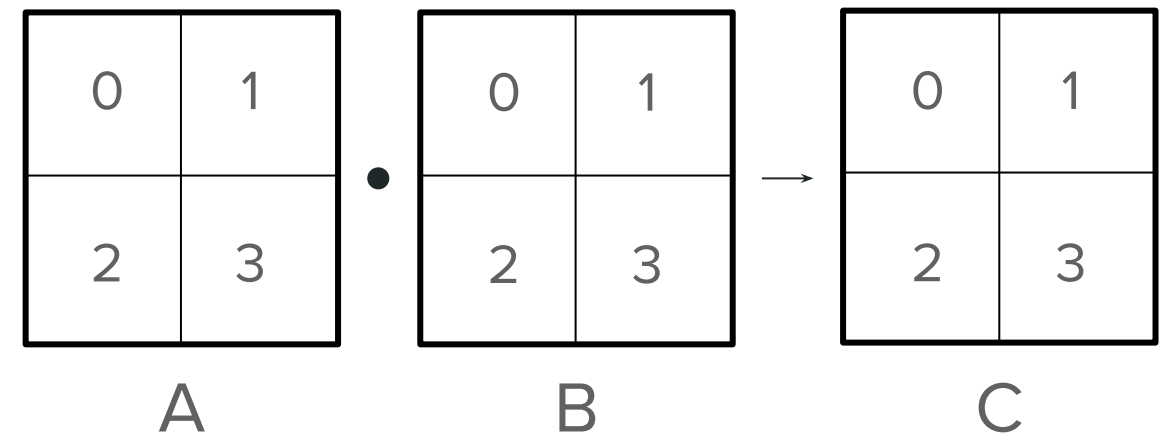


Distributed Matrix Multiplication

When a matrix is **too big** to fit
on a **single node/GPU**

When computation **takes too
long**

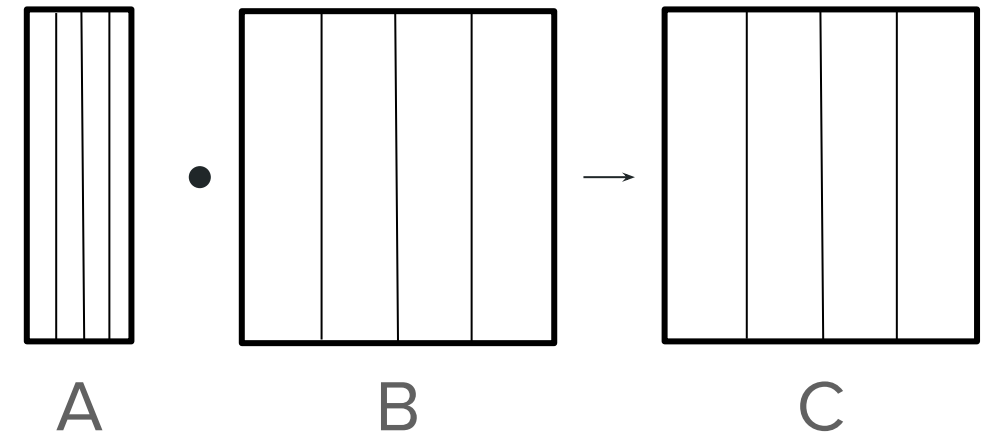
⇒ **Partition** matrix across
multiple GPUs



Distributed Matrix Multiplication

Different **problem sizes** require **different partitionings**

This is because **partitioning** determines **communication behavior**



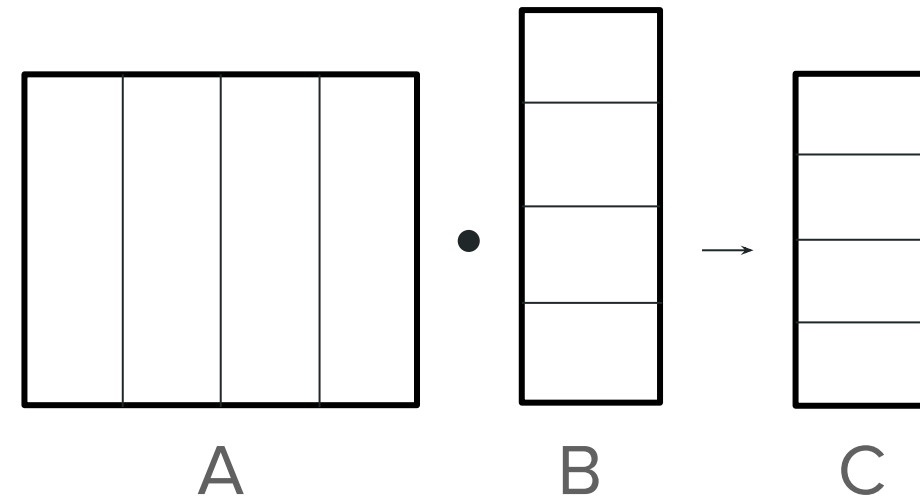
A is small $\Rightarrow$ pick a partitioning that only communicates A.

(1D column partitioning)

Distributed Matrix Multiplication

Different **problem sizes** require **different partitionings**

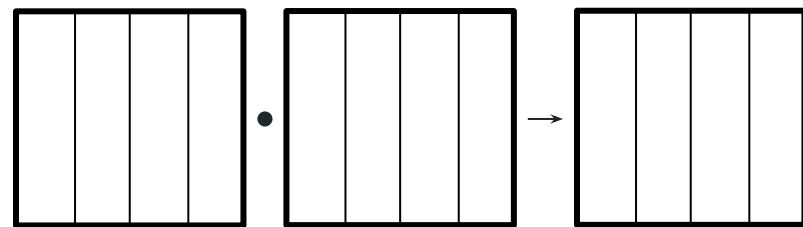
This is because **partitioning** determines **communication behavior**



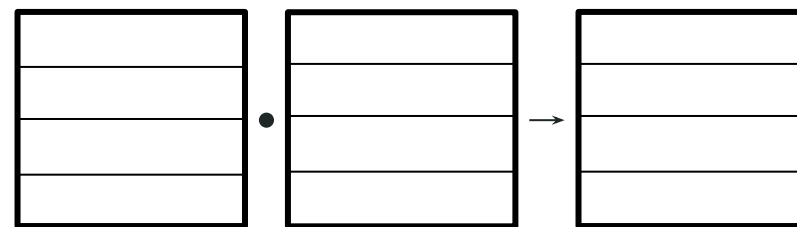
C is small $\Rightarrow$ pick a partitioning that only communicates C.

(Outer product partitioning)

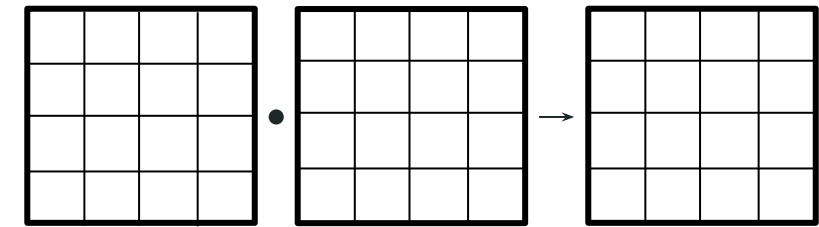
Flavors of Distributed Matrix Multiplication



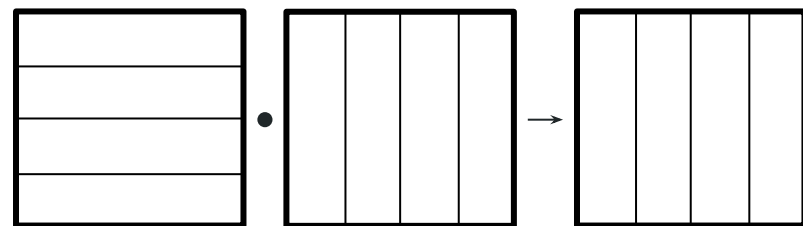
Column Block



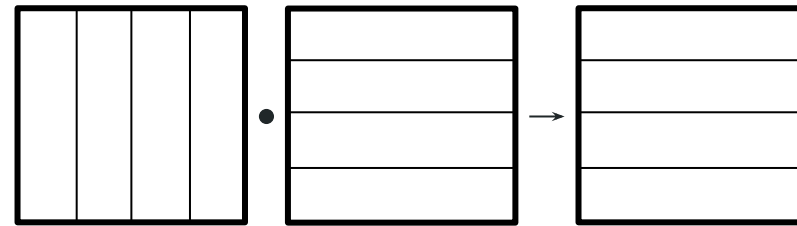
Row Block



2D Distribution



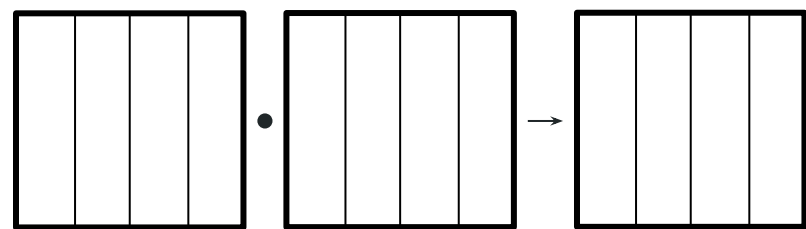
Inner Product



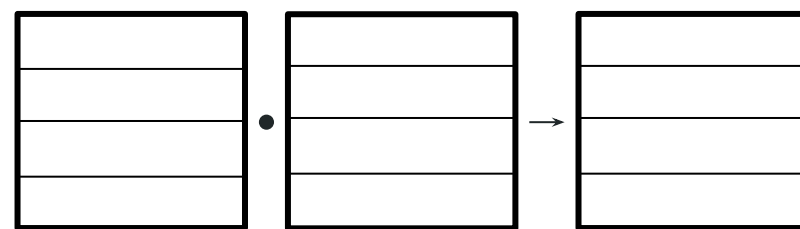
Outer Product

**+ cyclic distributions,
replication**

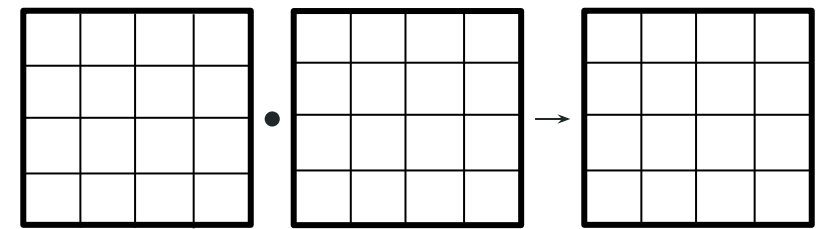
Flavors of Distributed Matrix Multiplication



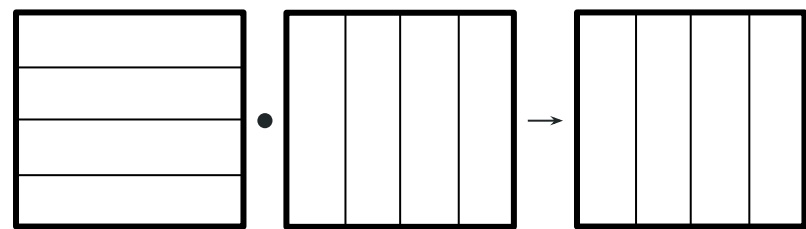
Column Block



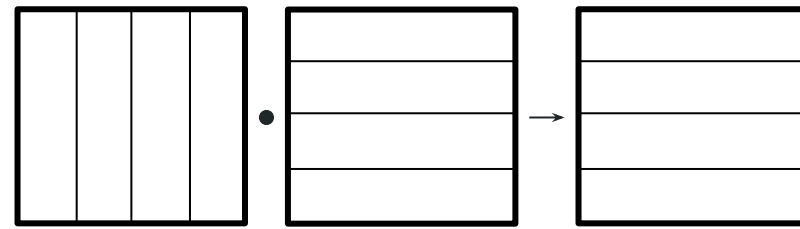
Row Block



2D Distribution



Inner Product



Outer Product

+ cyclic distributions,
replication

Different distributions suitable for **different problems**.

Must **implement distributed algorithm** for **each variant**.

SPMD Systems: Describe Partitionings

AI frameworks have developed “**SPMD**” systems, which allow users to set partitionings

SPMD systems then use **dispatching logic** to invoke an appropriate implementation

If no implementation is available, must **reshard** matrices

```
from torch.distributed.tensor import DeviceMesh,
Replicate, Shard, randn

# Create `a` and `b` with row block partitioning
a = randn((m,k),
          DeviceMesh('xpu', (world_size,)),
          [Shard(0)])
b = randn((k,n),
          DeviceMesh('xpu', (world_size,)),
          [Shard(0)])

# Perform distributed MatMul
c = torch.matmul(a, b)
```



SPMD Systems: Describe Partitionings

AI frameworks have developed “**SPMD**” systems, which allow users to set partitionings

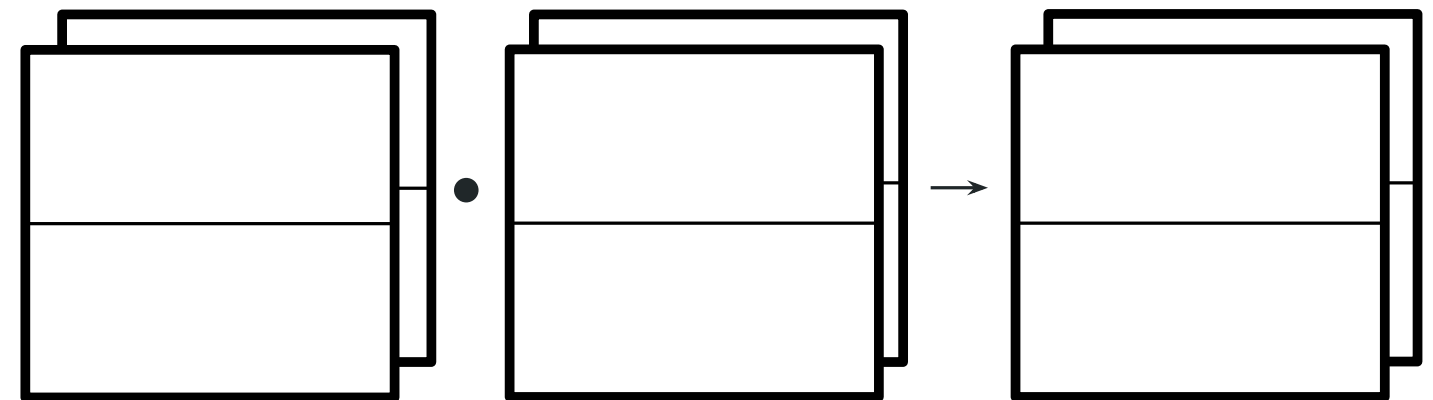
SPMD systems then use **dispatching logic** to invoke an appropriate implementation

If no implementation is available, must **reshard** matrices

```
from torch.distributed.tensor import DeviceMesh, Replicate, Shard, randn

# Create `a` and `b` with row block partitioning
a = randn((m,k),
          DeviceMesh('xpu', (2, world_size/2,)),
          [Replicate(), Shard(0)])
b = randn((k,n),
          DeviceMesh('xpu', (2, world_size/2,)),
          [Replicate(), Shard(0)])

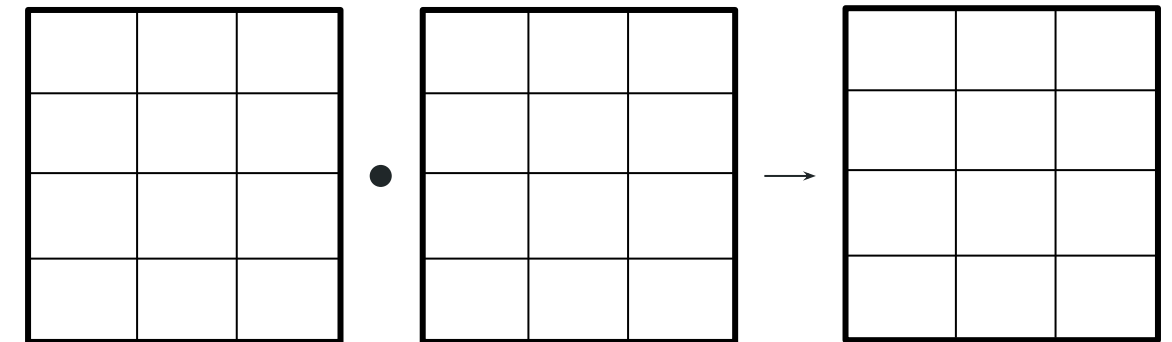
# Perform distributed MatMul
c = torch.matmul(a, b)
```



Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

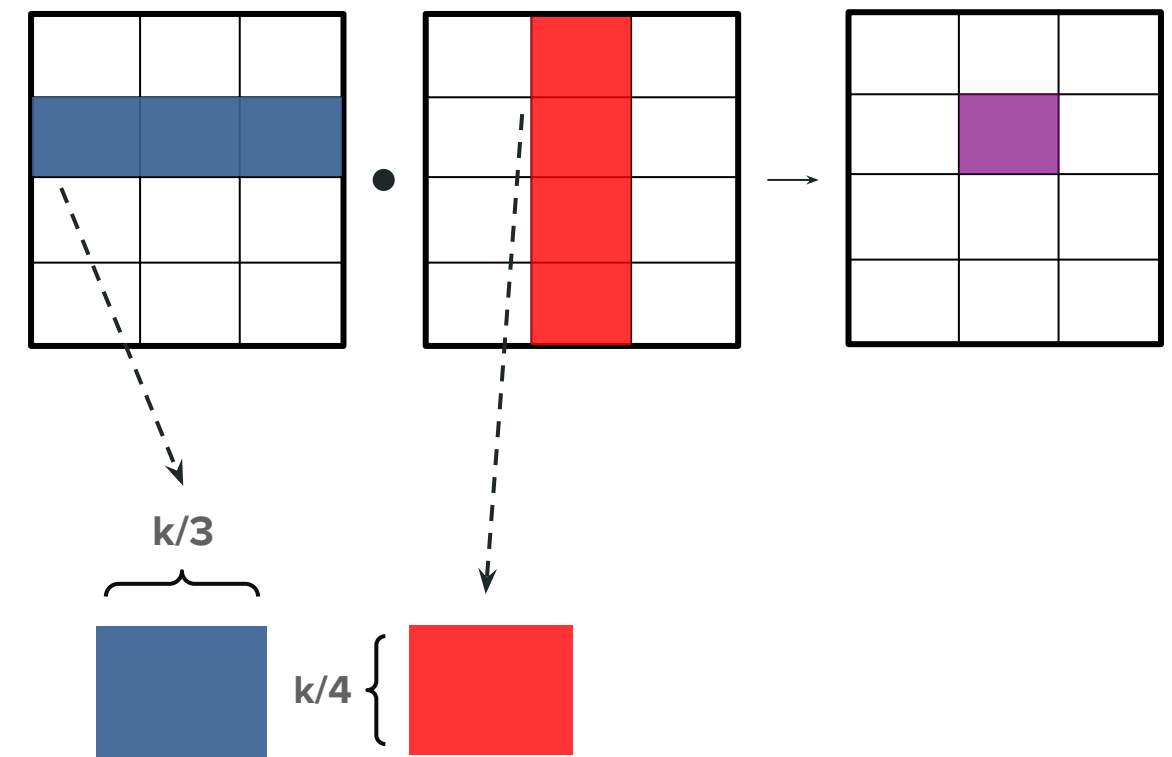
However, it doesn't work with all distributions.



Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

However, it doesn't work with all distributions.

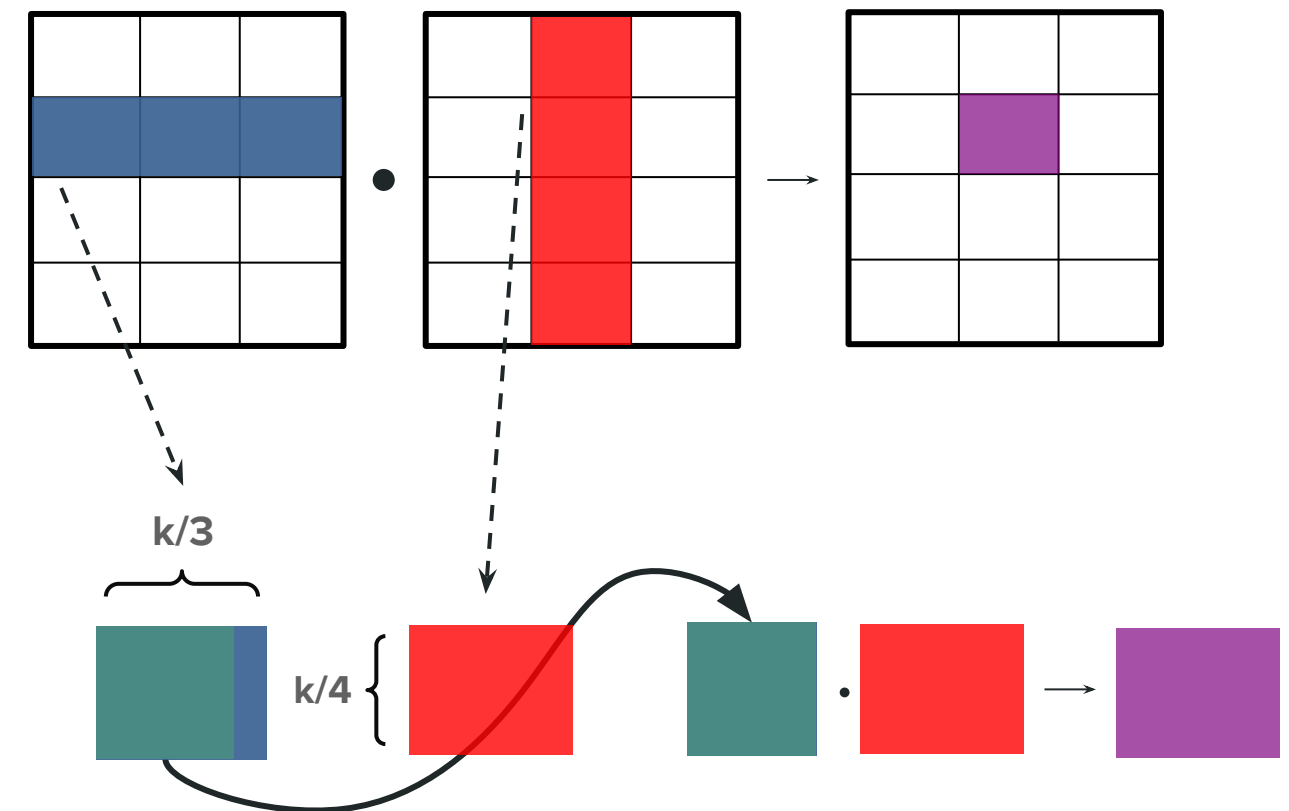


A is too wide! Dimensions incompatible.

Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

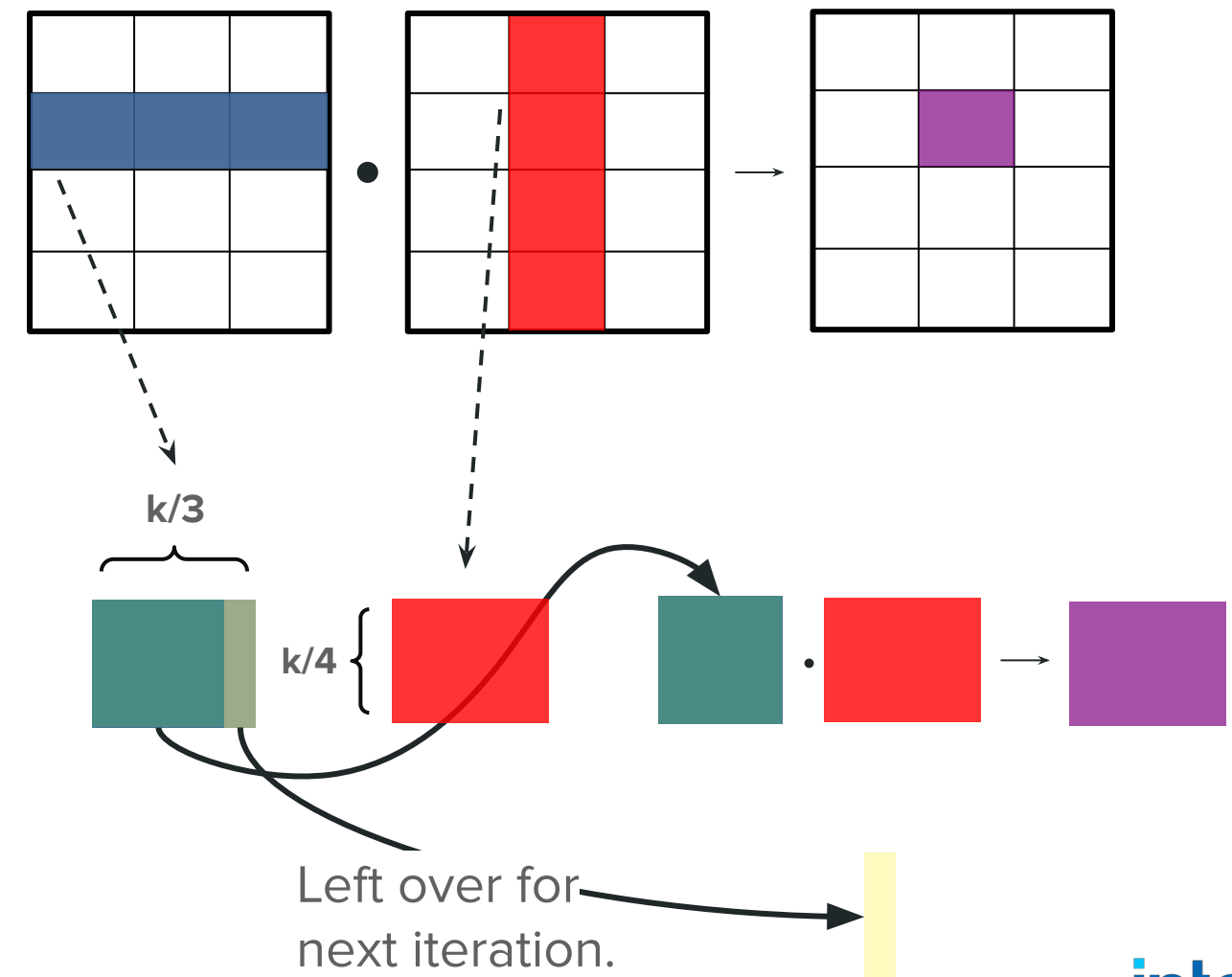
However, it doesn't work with all distributions.



Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

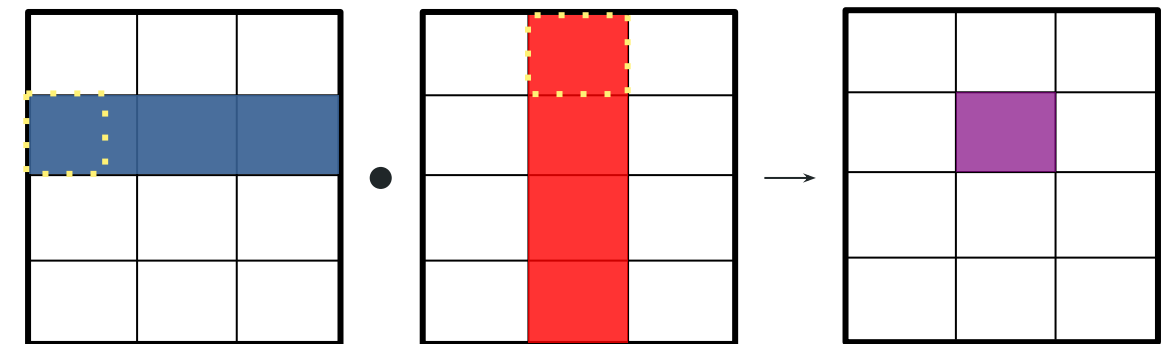
However, it doesn't work with all distributions.



Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

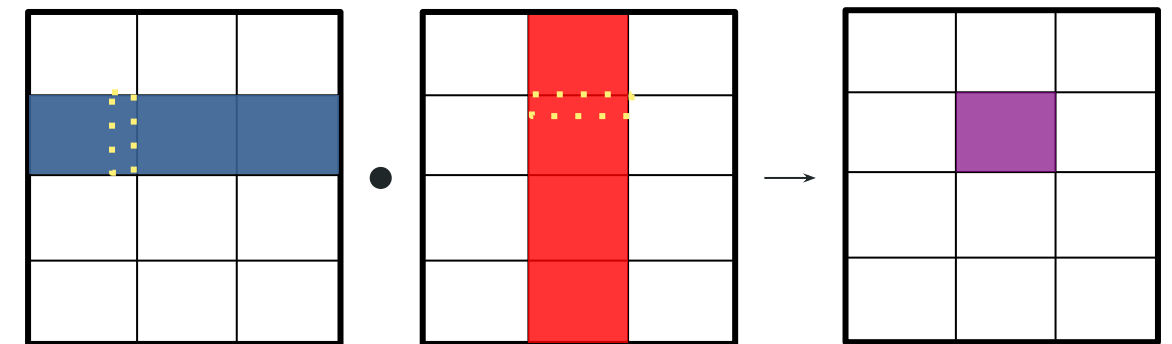
However, it doesn't work with all distributions.



Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

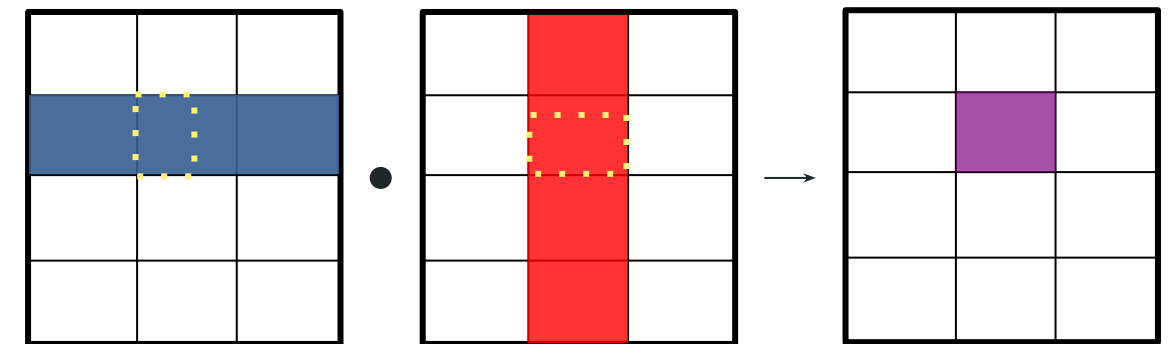
However, it doesn't work with all distributions.



Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

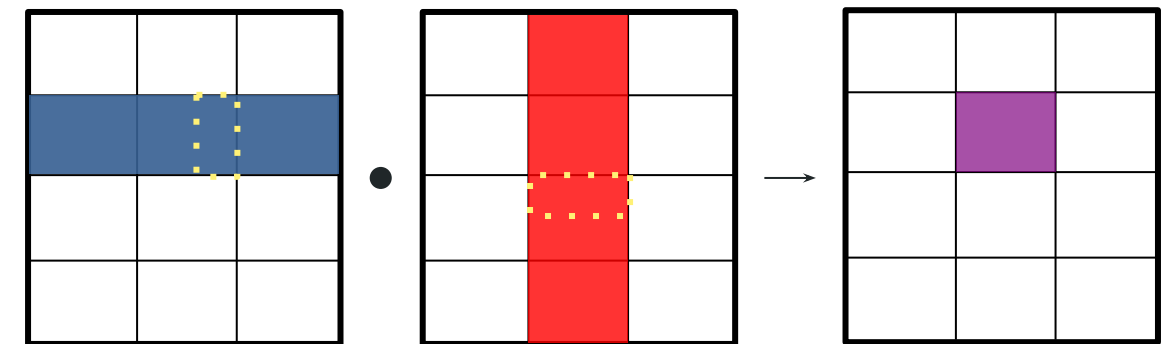
However, it doesn't work with all distributions.



Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

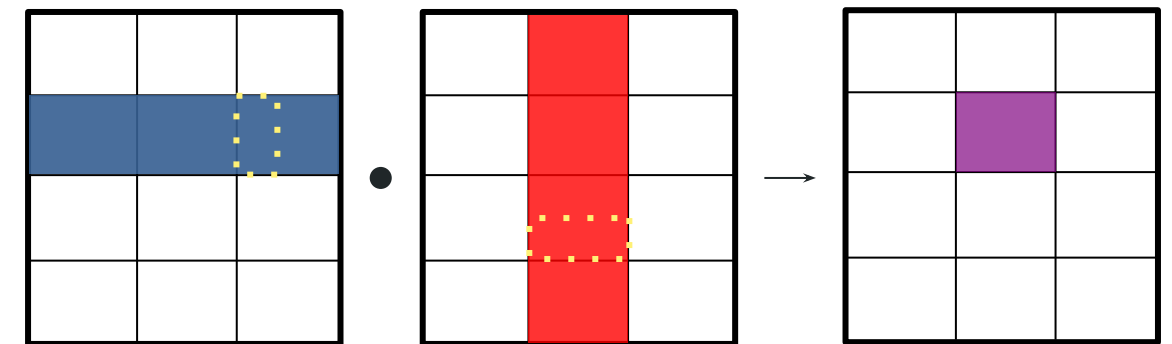
However, it doesn't work with all distributions.



Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

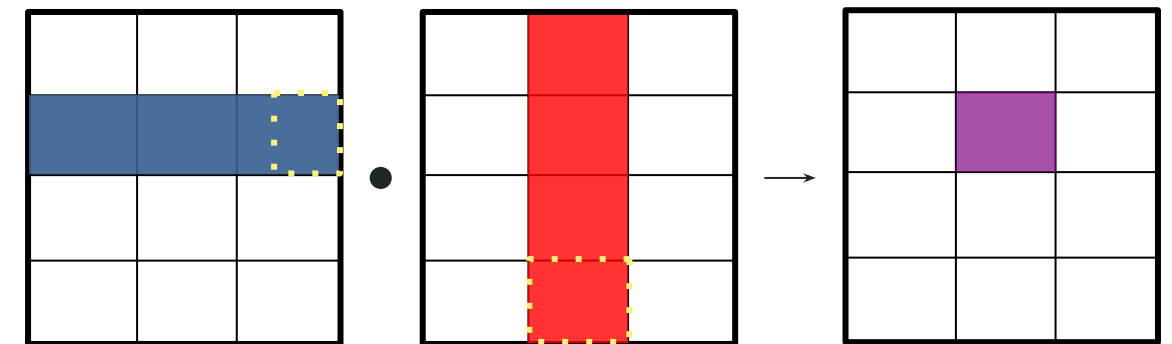
However, it doesn't work with all distributions.



Another Problem with SUMMA-Like Algorithms

SUMMA stands for “**Scalable Universal Matrix Multiplication Algorithm**”

However, it doesn't work with all distributions.



Universal RDMA Algorithm: Optimizations

- Instead of **naively fetching** and **multiplying** tiles, **construct computation graph** of tiles to be multiplied.

Pseudocode: Produce IR

```
# Each process produces a list of operations to perform.
for idx, block in 'blocks of C I own':
    # Compute the bounds of my tile
    c_bounds = c.tile_bounds(idx)

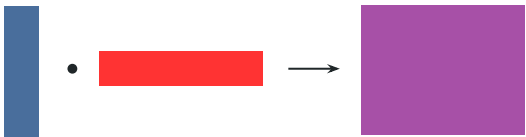
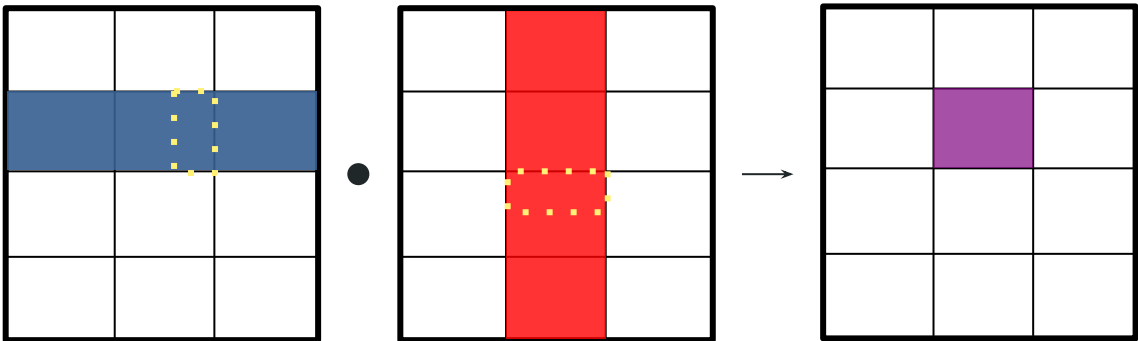
    # Find tiles of A that overlap.
    a_tiles = a.overlapping_tiles(c_bounds[0], :)

    for a_idx in a_tiles:
        # Find tiles of B that overlap with our output bounds (of C)
        # *And* the current A tile's bounds.
        a_tile_bounds = a.tile_bounds(a_idx)
        b_tiles = b.overlapping_tiles(a_tile_bounds[1], c_bounds[1])

        for b_idx in b_tiles:
            # Emit instruction that multiplies tiles of
            # A, B, and C, including the appropriate slicing
            # of these tiles.
```

Slicing-Based Universal Algorithm

- Instead of **naively fetching** and **multiplying** tiles, **construct computation graph** of tiles to be multiplied.



Rank 0

<code>C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]</code>

Universal RDMA Algorithm: Optimizations

- Instead of **naively fetching** and **multiplying** tiles, **construct computation graph** of tiles to be multiplied.
- This graph can be **reordered** and **modified** to implement **computation/communication overlap**, prevent fetching tiles multiple times.

Rank 0

<code>C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]</code>
<code>C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]</code>

Universal RDMA Algorithm: Optimizations

- Instead of naively fetching and multiplying tiles, **construct computation graph** of tiles to be multiplied.

Rank 0

C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]

a_tile = A.get_tile(0,0)
b_tile = B.get_tile(0,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:32] * b_tile[0:32,0:43]
a_tile = A.get_tile(0,0)
b_tile = B.get_tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,32:43] * b_tile[0:11,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:21] * b_tile[11:32,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,21:43] * b_tile[0:22,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:10] * b_tile[22:32,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(3,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,10:42] * b_tile[0:32,0:43]

Universal RDMA Algorithm: Optimizations

Redundant Communications

- Instead of repeatedly fetching and multiplying tiles, construct computation graph of tiles to be multiplied.

Rank 0

C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]

a_tile = A.get_tile(0,0)
b_tile = B.get_tile(0,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:32] * b_tile[0:32,0:43]
a_tile = A.get_tile(0,0)
b_tile = B.get_tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,32:43] * b_tile[0:11,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:21] * b_tile[11:32,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,21:43] * b_tile[0:22,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:10] * b_tile[22:32,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(3,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,10:42] * b_tile[0:32,0:43]

Universal RDMA Algorithm: Optimizations

Redundant Communications

- Instead of repeatedly fetching and multiplying tiles, construct computation graph of tiles to be multiplied.

Rank 0

C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]

a_tile = A.get_tile(0,0)
b_tile = B.get_tile(0,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:32] * b_tile[0:32,0:43]
a_tile = A.get_tile(0,0)
b_tile = B.get_tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,32:43] * b_tile[0:11,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:21] * b_tile[11:32,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,21:43] * b_tile[0:22,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:10] * b_tile[22:32,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(3,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,10:42] * b_tile[0:32,0:43]

Universal RDMA Algorithm: Optimizations

Redundant Communications

- Instead of repeatedly fetching and multiplying tiles, construct computation graph of tiles to be multiplied.

Rank 0

C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]

a_tile = A.get_tile(0,0)
b_tile = B.get_tile(0,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:32] * b_tile[0:32,0:43]
a_tile = A.get_tile(0,0)
b_tile = B.get_tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,32:43] * b_tile[0:11,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:21] * b_tile[11:32,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,21:43] * b_tile[0:22,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:10] * b_tile[22:32,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(3,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,10:42] * b_tile[0:32,0:43]

Universal RDMA Algorithm: Optimizations

Redundant Communications

- Instead of redundantly fetching and multiplying tiles, construct computation graph of tiles to be multiplied.

Rank 0

C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]

a_tile = A.get_tile(0,0)
b_tile = B.get_tile(0,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:32] * b_tile[0:32,0:43]
a_tile = A.get_tile(0,0)
b_tile = B.get_tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,32:43] * b_tile[0:11,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:21] * b_tile[11:32,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,21:43] * b_tile[0:22,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:10] * b_tile[22:32,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(3,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,10:42] * b_tile[0:32,0:43]

Universal RDMA Algorithm: Optimizations

Redundant Communications

- Instead of repeatedly fetching and multiplying tiles, construct computation graph of tiles to be multiplied.

Rank 0

C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]

a_tile = A.get_tile(0,0)
b_tile = B.get_tile(0,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:32] * b_tile[0:32,0:43]
a_tile = A.get_tile(0,0)
b_tile = B.get_tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,32:43] * b_tile[0:11,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:21] * b_tile[11:32,0:43]
a_tile = A.get_tile(0,1)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,21:43] * b_tile[0:22,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:10] * b_tile[22:32,0:43]
a_tile = A.get_tile(0,2)
b_tile = B.get_tile(3,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,10:42] * b_tile[0:32,0:43]

Universal RDMA Algorithm: Optimizations

1. Eliminate Redundant Communication

Rank 0

```
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]
```

```
a_tile = A.get_tile(0,0)
b_tile = B.get_tile(0,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:32] * b_tile[0:32,0:43]
```

```
b_tile = B.get_tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,32:43] * b_tile[0:11,0:43]
a_tile = A.get_tile(0,1)
```

```
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:21] * b_tile[11:32,0:43]
```

```
b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,21:43] * b_tile[0:22,0:43]
a_tile = A.get_tile(0,2)
```

```
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:10] * b_tile[22:32,0:43]
```

```
b_tile = B.get_tile(3,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,10:42] * b_tile[0:32,0:43]
```



Universal RDMA Algorithm: Optimizations

2. Prefetching

Rank 0

```
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,0:32] * B.tile(0,0)[0:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,0)[0:32,32:43] * B.tile(1,0)[0:11,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,0:21] * B.tile(1,0)[11:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,1)[0:32,21:43] * B.tile(2,0)[0:22,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,0:10] * B.tile(2,0)[22:32,0:43]
C.tile(0,0)[0:32,0:43] += A.tile(0,2)[0:32,10:42] * B.tile(3,0)[0:32,0:43]
```

```
a_tile = A.get_tile(0,0)
b_tile = B.get_tile(0,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:32] * b_tile[0:32,0:43]

b_tile = B.get_tile(1,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,32:43] * b_tile[0:11,0:43]
a_tile = A.get_tile(0,1)

C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:21] * b_tile[11:32,0:43]

b_tile = B.get_tile(2,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,21:43] * b_tile[0:22,0:43]
a_tile = A.get_tile(0,2)

C.tile(0,0)[0:32,0:43] += a_tile[0:32,0:10] * b_tile[22:32,0:43]

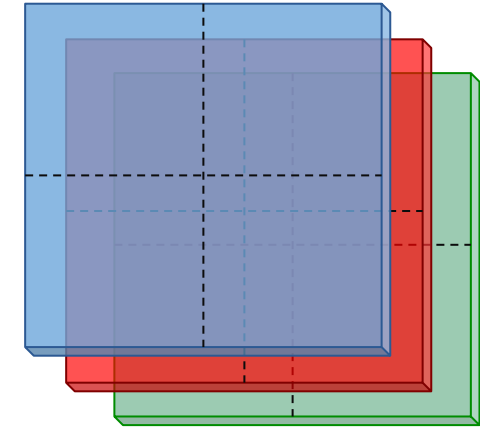
b_tile = B.get_tile(3,0)
C.tile(0,0)[0:32,0:43] += a_tile[0:32,10:42] * b_tile[0:32,0:43]
```

Universal RDMA Algorithm: Optimizations

- We can also support **replication** of one or more matrices.

Replication Factor $c = 3$
(3 Copies)

Each replica split
between 4 GPUs.

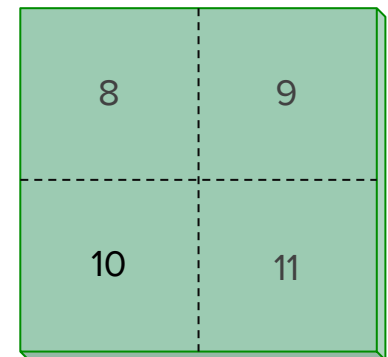
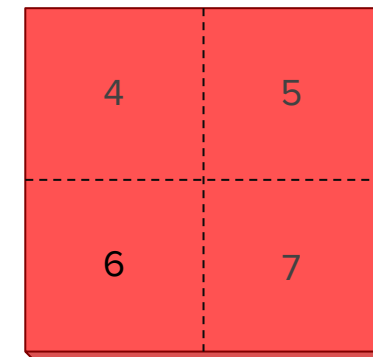
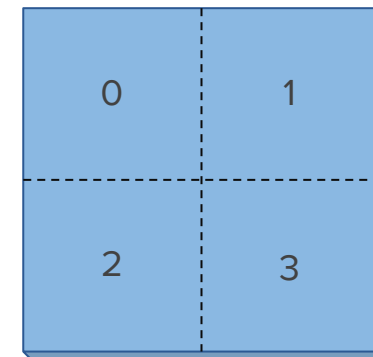
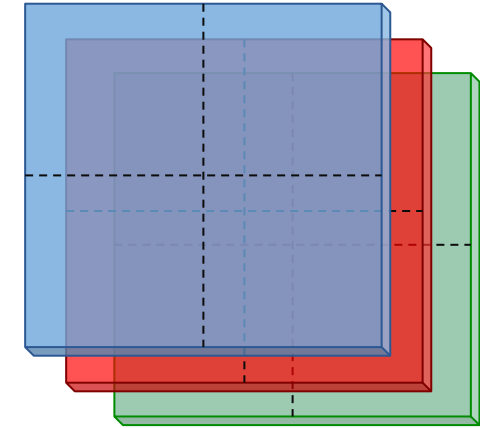


Universal RDMA Algorithm: Optimizations

- We can also support **replication** of one or more matrices.

Replication Factor $c = 3$
(3 Copies)

Each replica split
between 4 GPUs.

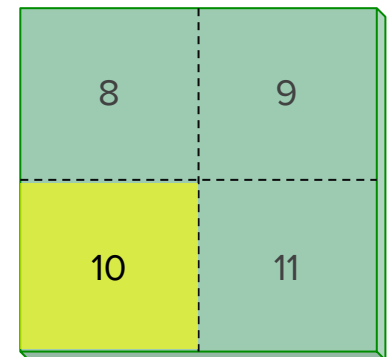
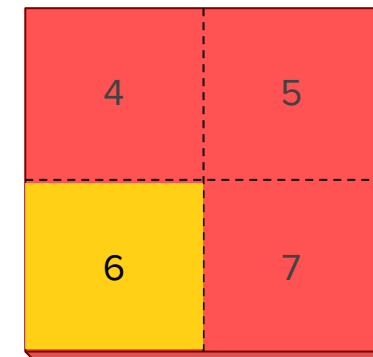
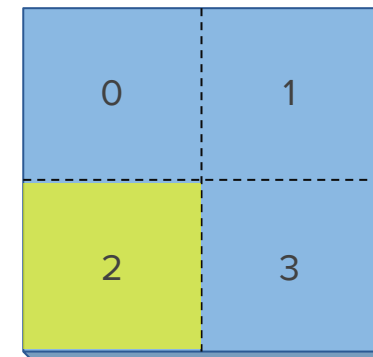
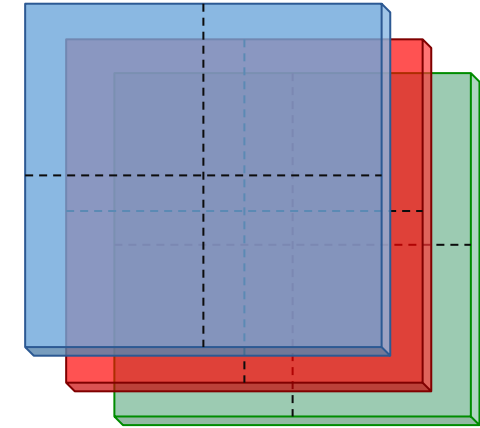


Universal RDMA Algorithm: Optimizations

- We can also support **replication** of one or more matrices.

Replication Factor $c = 3$
(3 Copies)

Each replica split
between 4 GPUs.

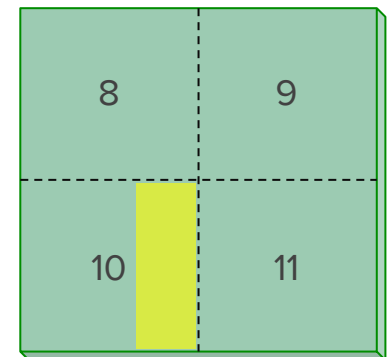
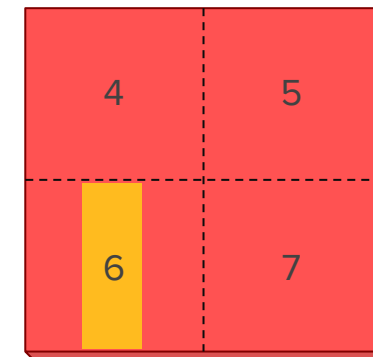
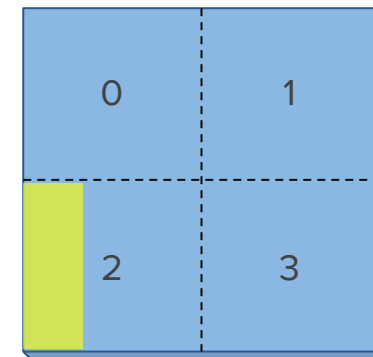
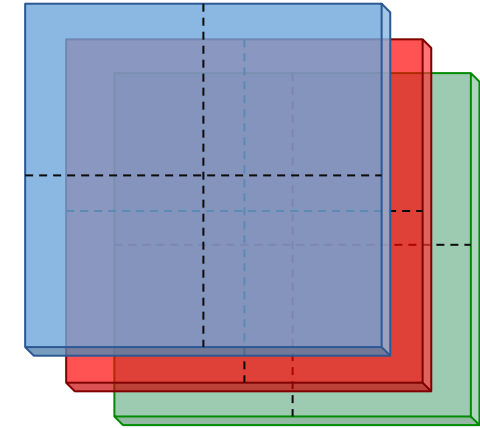


Universal RDMA Algorithm: Optimizations

- We can also support **replication** of one or more matrices.
- This requires **decomposing** the inner loop (split-k style).

Replication Factor $c = 3$
(3 Copies)

Each replica split
between 4 GPUs.

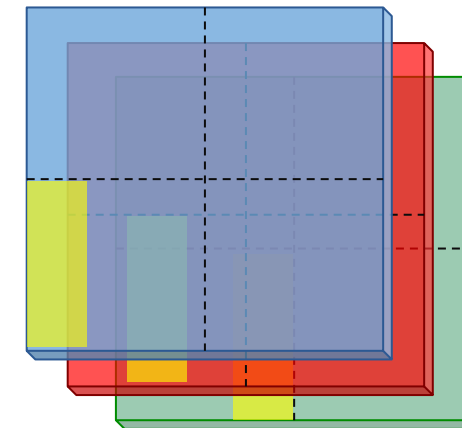
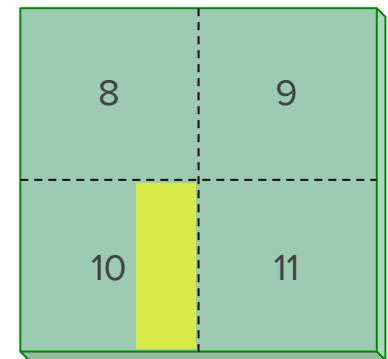
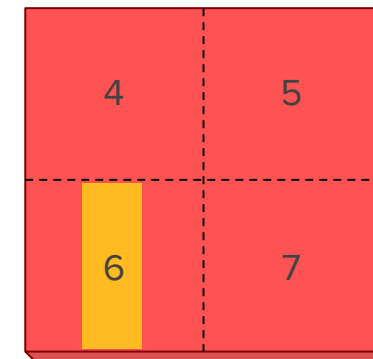
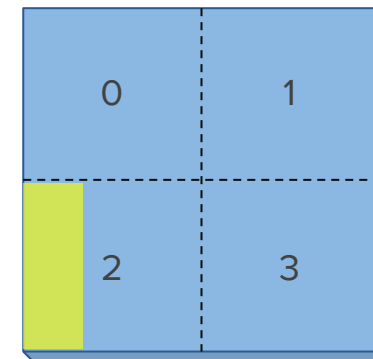
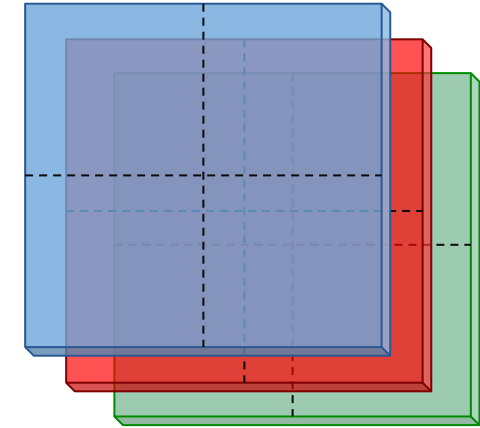


Universal RDMA Algorithm: Optimizations

- We can also support **replication** of one or more matrices.
- This requires **decomposing** the inner loop (split-k style).

Replication Factor $c = 3$
(3 Copies)

Each replica split
between 4 GPUs.

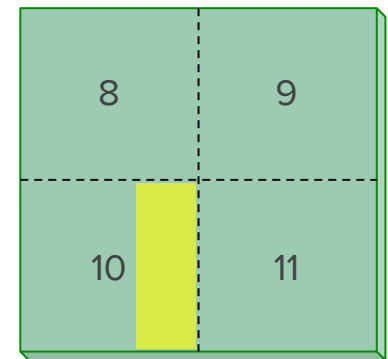
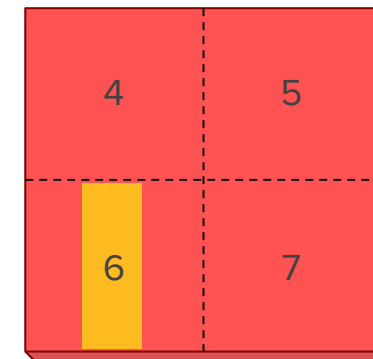
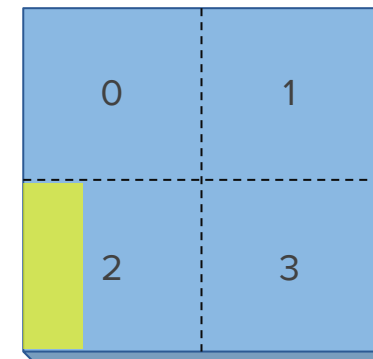
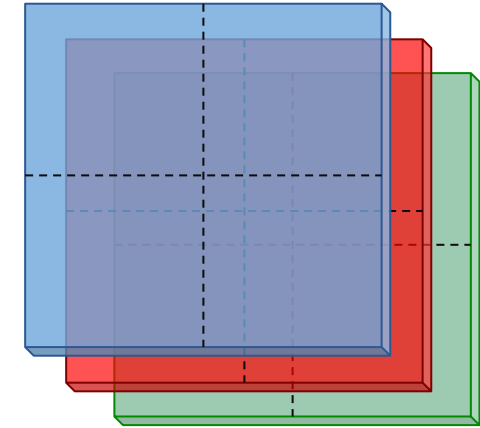


Universal RDMA Algorithm: Optimizations

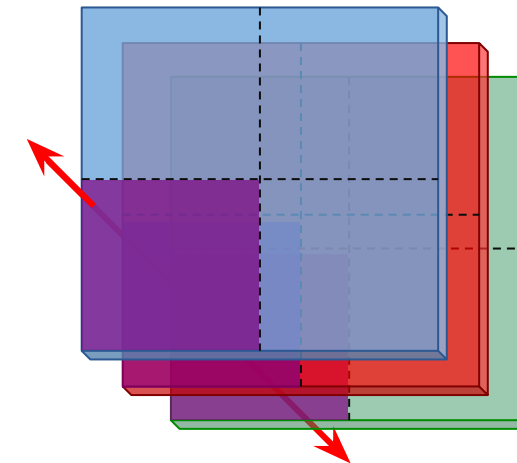
- We can also support **replication** of one or more matrices.
- This requires decomposing the inner loop (split-k style).
- Followed by an **AllReduce** to accumulate the final result.

Replication Factor $c = 3$
(3 Copies)

Each replica split
between 4 GPUs.



AllReduce!



Universal RDMA Algorithm: Replication

- Generating IR for replicated matrix multiplication is **trivial** under this model.
- Simply **split the iteration space** based on the **replication factor**.

Pseudocode: Produce IR

```
# Each process produces a list of operations to perform.
for idx, block in 'blocks of C I own':
    # Compute the bounds of my tile
    c_bounds = c.tile_bounds(idx)

    # Find tiles of A that overlap.
    a_tiles = a.overlapping_tiles(c_bounds[0], :)

    for a_idx in a_tiles:
        # Find tiles of B that overlap with our output bounds (of C)
        # *And* the current A tile's bounds.
        a_tile_bounds = a.tile_bounds(a_idx)
        b_tiles = b.overlapping_tiles(a_tile_bounds[1], c_bounds[1])

        for b_idx in b_tiles:
            # Emit instruction that multiplies tiles of
            # A, B, and C, including the appropriate slicing
            # of these tiles.
```

Universal RDMA Algorithm: Replication

- Generating IR for replicated matrix multiplication is **trivial** under this model.
- Simply **split the iteration space** based on the **replication factor**.

Pseudocode: Produce IR

```
# Each process produces a list of operations to perform.
for idx, block in 'blocks of C I own':
    # Compute the bounds of my tile
    c_bounds = c.tile_bounds(idx)

    # Find tiles of A that overlap.
    my_split_k = slice(my_replica_index*slice_size,
                      (my_replica_index+1)*slice_size)
    a_tiles = a.overlapping_tiles(c_bounds[0], my_split_k)

    for a_idx in a_tiles:
        # Find tiles of B that overlap with our output bounds (of C)
        # *And* the current A tile's bounds.
        a_tile_bounds = a.tile_bounds(a_idx)
        b_tiles = b.overlapping_tiles(a_tile_bounds[1], c_bounds[1])

        for b_idx in b_tiles:
            # Emit instruction that multiplies tiles of
            # A, B, and C, including the appropriate slicing
            # of these tiles.
```

Universal RDMA Algorithm: Replication

- Generating IR for replicated matrix multiplication is **trivial** under this model.
- Simply **split the iteration space** based on the **replication factor**.
- If **A** and **B** are replicated, **grab from local replica** (otherwise, just retrieve).

Pseudocode: Produce IR

```
# Each process produces a list of operations to perform.
for idx, block in 'blocks of C I own':
    # Compute the bounds of my tile
    c_bounds = c.tile_bounds(idx)

    # Find tiles of A that overlap.
    my_split_k = slice(my_replica_index*slice_size,
                      (my_replica_index+1)*slice_size)
    a_tiles = a.overlapping_tiles(c_bounds[0], my_split_k)

    for a_idx in a_tiles:
        # Find tiles of B that overlap with our output bounds (of C)
        # *And* the current A tile's bounds.
        a_tile_bounds = a.tile_bounds(a_idx)
        b_tiles = b.overlapping_tiles(a_tile_bounds[1], c_bounds[1])

        for b_idx in b_tiles:
            # Emit instruction that multiplies tiles of
            # A, B, and C, including the appropriate slicing
            # of these tiles.
```

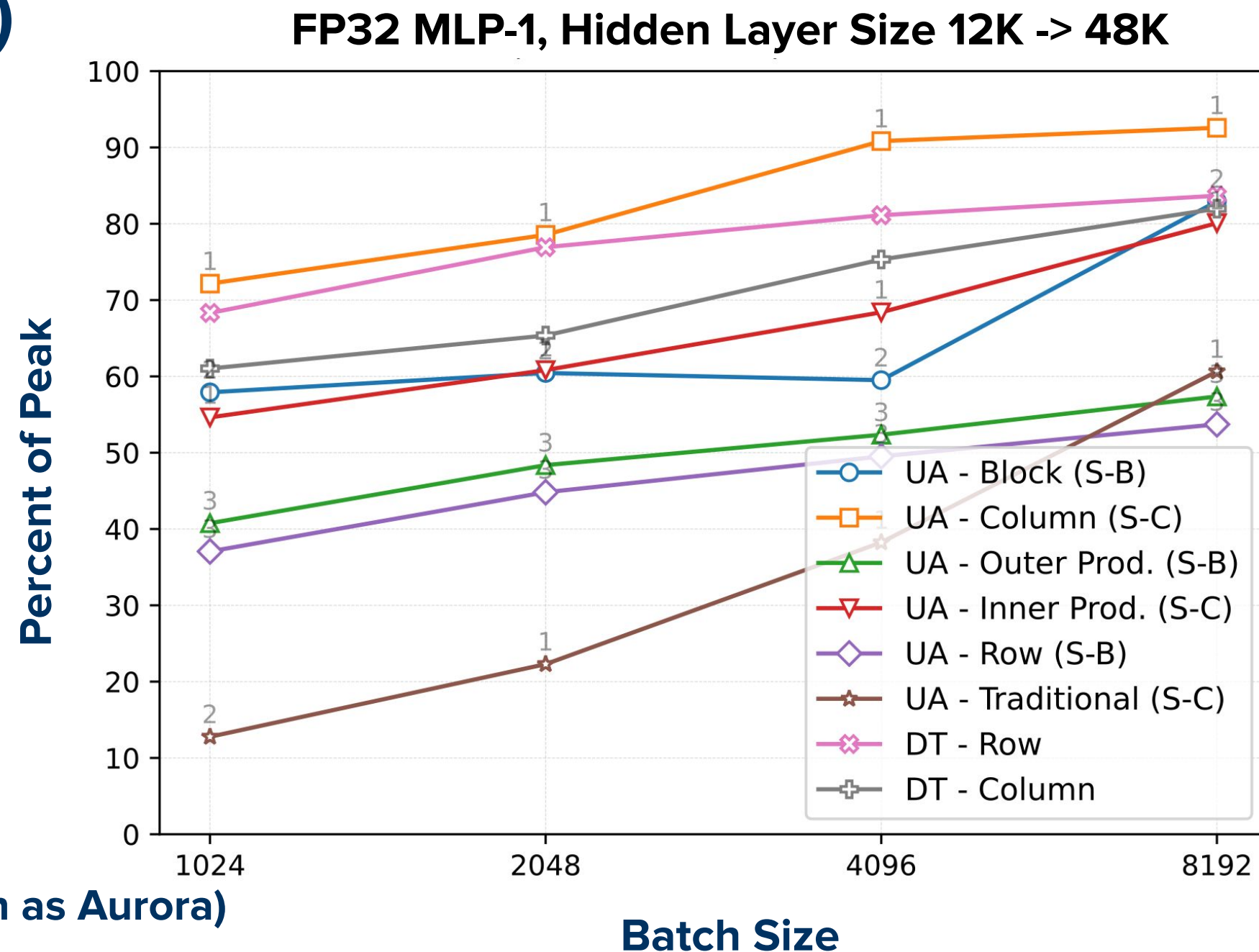

Transformer Layer MLP (1)

- We can set up problems with a **variety of distributions** and feed all of them into the universal algorithm.
- Different problem sizes favor different distributions.

Run on Borealis (Same Configuration as Aurora)

12 PVC Tiles per Node

20 GB/s Xe Link Between Tiles



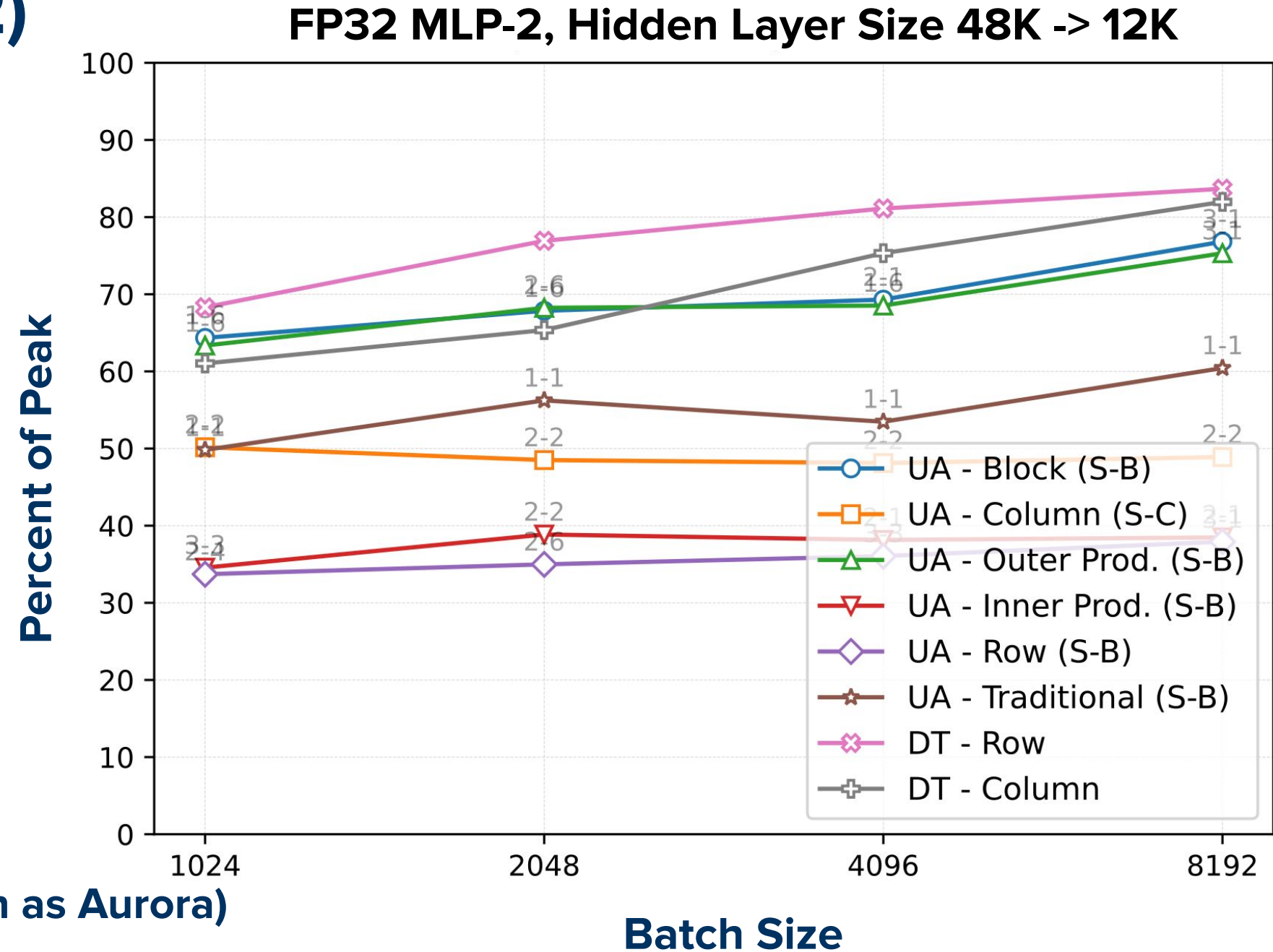
Transformer Layer MLP (2)

- We can set up problems with a **variety of distributions** and feed all of them into the universal algorithm.
- Different problem sizes favor different distributions.

Run on Borealis (Same Configuration as Aurora)

12 PVC Tiles per Node

20 GB/s Xe Link Between Tiles



Transformer Layer MLP (1)

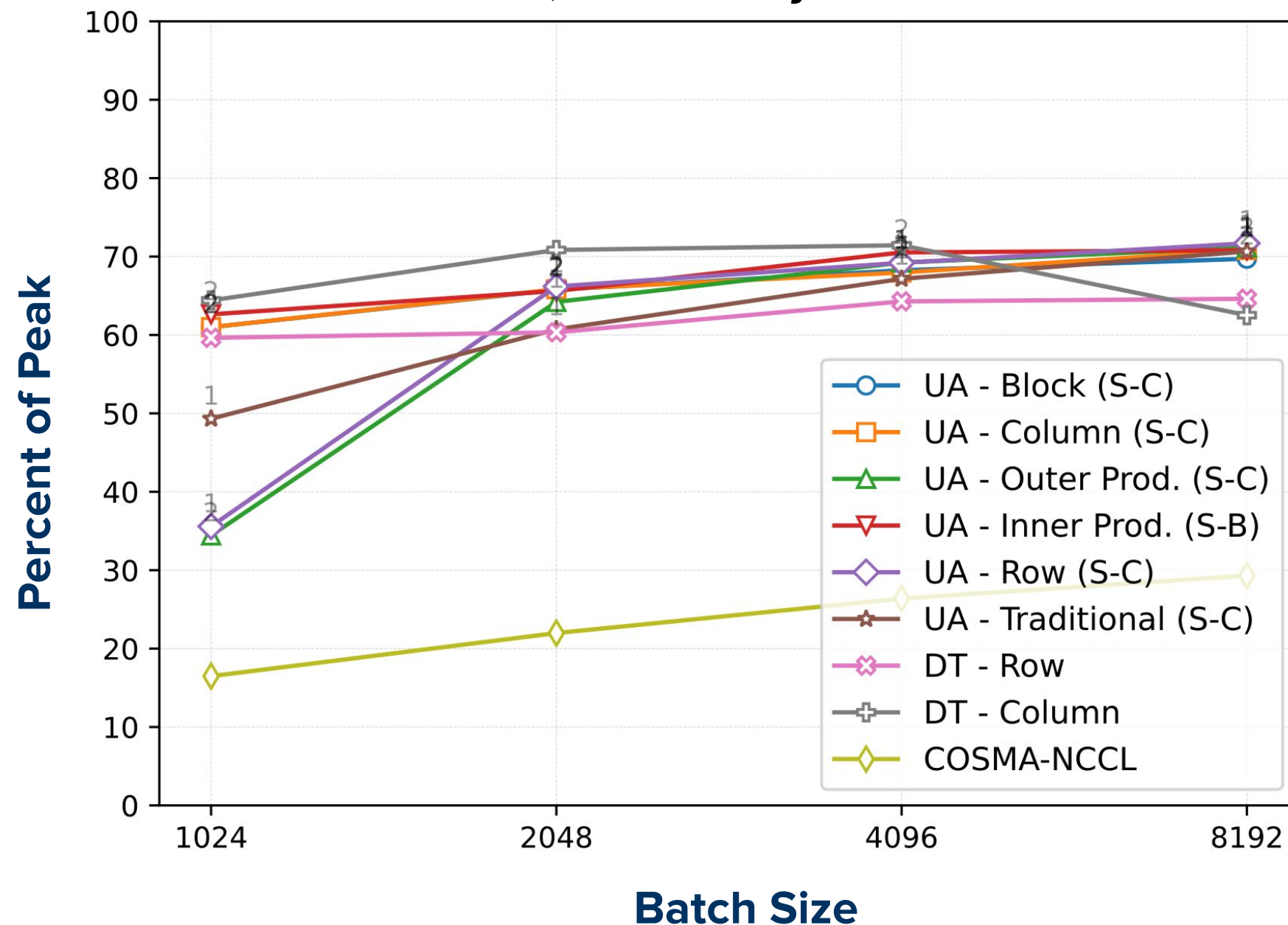
- We can set up problems with a **variety of distributions** and feed all of them into the universal algorithm.
- Different problem sizes favor different distributions.

Run on H100 System

8 H100 GPUs per Node

450 GB/s NVLink Between Tiles

FP32 MLP-1, Hidden Layer Size 12K -> 48K



Transformer Layer MLP (2)

- We can set up problems with a **variety of distributions** and feed all of them into the universal algorithm.
- Different problem sizes favor different distributions.

Run on H100 System

8 H100 GPUs per Node

450 GB/s NVLink Between Tiles

FP32 MLP-2, Hidden Layer Size 48K -> 12K

